Atividade — Animação 2D quadro a quadro com sprites em PPM

Nesta atividade, você irá aplicar a função `carimba` desenvolvida nas aulas anteriores para compor cenas gráficas a partir de um **cenário (background)** e um **sprite**, gerando uma sequência de **quadros (frames)** que, ao final, serão convertidos em um vídeo. O objetivo é consolidar os conceitos de composição de imagens, uso de canal alfa/transparência e manipulação do formato PPM, além de introduzir um fluxo de trabalho básico de produção de animações 2D quadro a quadro (**frame-by-frame**).

A função `carimba`

A função `carimba(destino, origem, x, y, usar_transparencia, cor_transparente)` (ou equivalente implementada por você) deve ser capaz de:

- Sobrepor uma imagem *origem* (o **sprite**) sobre uma imagem *destino* (o cenário ou o frame anterior), posicionando-a nas coordenadas (x, y);
- Respeitar os limites da imagem de destino, descartando ou recortando pixels que ultrapassem a borda;
- Quando `usar_transparencia` estiver habilitado, ignorar (não copiar) os pixels do sprite que correspondam à cor definida como transparente (**chroma key**, geralmente um verde ou magenta puro, ou pixels com canal

alfa em imagens RGBA), preservando o conteúdo do destino nesses pontos.

Essa função será a base para "carimbar" cada sprite de uma animação sobre o cenário, quadro a quadro, gerando a ilusão de movimento.

Materiais

Você deverá escolher:

- Um **cenário (fundo estático)** de sua preferência;
- **Sprites** de personagens/objetos, obtidos de um dos seguintes sites:
  - https://www.spriters-resource.com/
  - https://spritedatabase.net/

Muitos sprites nesses sites são disponibilizados como **sprite sheet** (uma única imagem contendo vários quadros organizados em grade). É permitido (e

recomendado) baixar a sprite sheet e fatiá-la programaticamente em quadros individuais, desde que isso seja feito de forma **automática (por código)**, e não manualmente em um editor de imagens.

O programa deve carregar o cenário e a(s) sprite sheet(s) automaticamente a partir de arquivos de entrada PPM convertidos previamente, sem intervenção manual durante a execução.

Caso a sprite sheet contenha múltiplos quadros em grade, o código deve recortar cada quadro automaticamente com base nas dimensões (largura/altura de cada célula), gerando uma lista/array de sprites individuais.

Geração dos frames

Para cada quadro da animação, o programa deve:

1. Partir de uma cópia do cenário de fundo;
2. Usar a função `carimba` para posicionar o sprite correspondente àquele instante da animação sobre o cenário, com transparência habilitada;
3. Salvar o resultado como um arquivo PPM sequencial (ex.: `frame_000.ppm`, `frame_001.ppm`, `frame_002.ppm`, ...).

O formato de saída deve ser **PPM**, preferencialmente no formato binário **P6**; o formato texto (**P3**) é aceito caso haja dificuldade de implementação do P6, mas será valorizado o uso do P6.

A movimentação do sprite ao longo dos frames (troca de pose/quadro da sprite sheet e/ou deslocamento de posição x, y no cenário) deve ser clara e produzir uma animação fluida e reconhecível.

Geração do vídeo

Após gerar todos os frames em PPM, você deverá utilizar o **ffmpeg** (ou ferramenta equivalente) para compor os frames em um arquivo de vídeo (ex.: .mp4 ou .gif).

Exemplo de comando de referência (ajustar taxa de quadros, nomenclatura e resolução conforme o projeto):

```
ffmpeg -framerate 12 -i
frame_%03d.ppm -c:v libx264 -
pix_fmt yuv420p animacao.mp4
```

O comando (ou script) utilizado para gerar o vídeo deve ser documentado no relatório de entrega.

Entregáveis: três animações distintas

Deverão ser produzidas três animações distintas, atendendo aos critérios abaixo:

Animação 1 (movimento em loop): um sprite realizando um ciclo de movimento simples sobre um cenário fixo (ex.: caminhada, corrida ou ataque de um personagem, em looping).

Animação 2 (deslocamento na cena): um sprite diferente do da Animação 1, podendo usar um cenário igual ou diferente, mas com um tipo de movimento distinto: deve haver deslocamento pela cena, além da troca de pose.

Animação 3 (tema livre): livre, à escolha do grupo, desde que utilize sprites/cenário diferentes das anteriores e demonstre algum elemento adicional (ex.: dois ou mais sprites carimbados na mesma cena, mudança de profundidade/ordem de sobreposição, ou variação de velocidade da animação).

Desafio: inclua o efeito de **paralaxe** em alguma das animações.

Cada uma das três animações deve gerar seu próprio conjunto de frames PPM e seu próprio vídeo final.

Entrega

- Código-fonte completo e comentado (incluindo a função `carimba` reutilizada/adaptada);
- Os três vídeos gerados (ou os frames PPM + script de geração do vídeo, caso o vídeo não possa ser anexado);
- Um pequeno relatório (1 a 2 páginas) contendo:
  - Sprites e cenários utilizados, com os links de origem;
  - Explicação de como a transparência foi tratada para cada sprite;

    - Comando(s) utilizado(s) no ffmpeg para cada animação;
    - Dificuldades encontradas e decisões de projeto.
- Créditos aos autores originais dos sprites/cenários devem constar no relatório, respeitando os termos de uso dos sites indicados.

Dica: gerando nomes numerados automaticamente

```
#include <stdio.h>

char nome_arquivo[64];
int frame;

for (frame = 0; frame <
total_frames; frame++) {
    // gera o nome:
frame_000.ppm, frame_001.ppm, ...
```

```
    snprintf(nome_arquivo,
sizeof(nome_arquivo),
"frame_%03d.ppm", frame);

    FILE *f = fopen(nome_arquivo,
"wb"); // "wb" para P6 (binário),
"wt" para P3 (ASCII)
    if (!f) {
        printf("Erro ao criar
%s\n", nome_arquivo);
        exit(-42);
    }

    // ... escreva o cabeçalho
PPM e os pixels do frame aqui ...

    fclose(f);
}
```

Explicação:

- `%03d` garante 3 dígitos com zero à esquerda (000, 001, ..., 999), o que mantém os arquivos em ordem alfabética/numérica — essencial para

o `ffmpeg -i frame_%03d.ppm` funcionar corretamente.

- Se você espera mais de 999 frames, aumente para `%04d` (e ajuste o padrão `%04d` no comando do ffmpeg também).
- Use `"wb"` (modo binário) ao abrir o arquivo se estiver gravando P6, já que os dados de pixel são bytes puros e não texto; em caso de P3 utilize `"wt"`.